Exact PTO Equivalence Binary Addition

Metric-Extracted Controller

`analyses/metric_extraction/DFST/add2/warmup_50000_steps_250000/`

1 Statement and Scope

Controllers. The exported table `minimized_transition_table.pdf` defines controller C, with initial state 0 and halt state 21. The function `add_grid(2)` defines expert $\mathcal{E}$, with initial state 0 and halt state 10. The adjacent expert export matches this function entry for entry. All tables retain the exported state numbers.

Input Domain. Let $D = \{0, 1\}$. Row 0 initially contains the contiguous word $a + b$, starting at column 0, with $a \in D^m$, $b \in D^n$, and $m, n \geq 1$. All remaining cells contain blanks. The head starts at $(0, 0)$. Write $A = \mathrm{val}_2(a)$ and $B = \mathrm{val}_2(b)$. The domain permits arbitrary operand lengths, unequal lengths, and leading zeros.

Execution Convention. The grid uses $\mathbb{Z}^2$, with coordinates increasing rightward and downward. The symbol λ denotes a blank. A transition (q', w, M) writes w, performs motion M, and enters state q'. Motions L, R, D, U move one cell. Motion S writes and stops without moving.

Policy-Trajectory Observations. For input $a + b$, define the policy-trajectory observation (PTO) by

$$\mathrm{PTO}(a + b) = \big((r_t, w_t, M_t)\big)_{t=0}^{T-1},$$

where r_t, w_t, and M_t denote the read symbol, written symbol, and motion at controller step t. A finite execution contributes the final S action exactly once, without halt-loop padding.

Theorem 1 (Exact PTO Equivalence)**.** *For every input in the stated domain,* $\mathrm{PTO}_C(a + b) = \mathrm{PTO}_{\mathcal{E}}(a + b)$. *Both executions terminate on the same step. The complete grids and head positions agree at every step, including the final configuration.*

The theorem concerns autonomous executions of the exported discrete controller on the stated input domain.

2 Expert Behavior

2.1 Accumulator and Marker Invariant

Write $B = \sum_{j=0}^{n-1} b_j 2^j$, with b_0 the rightmost digit. The expert scans to the separator at column m, steps left and down, writes a working + below column $m - 1$, and enters state 8 above the marker.

Lemma 1 (Round Invariant)**.** *After processing k right-operand digits, $0 \le k \le n$, the expert enters state 8 at $(p_k, 0)$, where $p_k = m - 1 - k$. At this point:*

- *Row 1 contains exactly one nonblank symbol: a + at column p_k.*
- *Row 0 contains a contiguous accumulator on $[\ell_k, m - 1]$, with value $C_k = A + \sum_{j<k} b_j 2^j$ and left edge $\ell_k \le p_k + 1$. Column $m - 1$ carries the units digit.*
- *The separator remains at column m. The unprocessed prefix of b occupies columns $m + 1, \ldots, m + n - k$. All remaining cells contain blanks.*

Proof. Initialization gives $k = 0$, $\ell_0 = 0$, $C_0 = A$, and $p_0 = m - 1$. State 8 preserves the digit at p_k, or writes 0 on a blank. A blank at p_k forces $\ell_k = p_k + 1$, so zero extension preserves contiguity.

For $k < n$, the right scan reaches the blank after the unprocessed prefix, steps left to digit b_k at $r_k = m + n - k$, erases b_k, and descends. The expert transports b_k left along row 1 to the marker. The distance satisfies

$$r_k - p_k = n + 1 \ge 2. \tag{1}$$

Consequently, the first two transport steps read blanks. At the marker, the expert ascends and adds b_k to the accumulator digit e at column p_k. If $e + b_k < 2$, the expert writes $e + b_k$ and descends. Otherwise, the expert writes $e + b_k - 2$, moves left, and propagates one carry. Each digit 1 changes to 0; the first smaller digit increases by one, or a blank changes to 1. The update adds exactly $b_k 2^k$ and preserves contiguity.

The expert returns along row 1, erases the marker, steps left, writes the next marker at $p_{k+1} = p_k - 1$, and ascends into state 8. The earlier zero extension gives $\ell_{k+1} \le p_k = p_{k+1} + 1$. The accumulator now has value C_{k+1}, establishing the next round. □

Lemma 2 (Termination and Final Configuration)**.** *The expert terminates with only the padded base-2 representation of $A + B$ on row 0, ending at column $m - 1$. The final head stops at $(m - n - 1, 0)$. Writing $\ell_2(s)$ for the binary digit count, with $\ell_2(0) = 1$, the output width equals $L = \max\{m, n + 1, \ell_2(A + B)\}$.*

Proof. Each round consumes one of the n right-operand digits. Every scan crosses a finite interval, and every carry reaches a digit below 1 or a blank. After round n, state 8 ensures a digit at p_n. The next right scan returns to the separator. State 3 erases the separator and descends. State 9 walks left, erases the final marker, ascends, preserves the accumulator digit, and stops. The invariant gives $C_n = A + B$, and cleanup removes all work symbols. The original m columns persist, and marker extension reaches p_n, ensuring at least $n + 1$ digits. Every additional carry extension contributes a digit required by the current numerical value. Partial sums never decrease, so the final width equals L. □

3 Observation Protocol

The following phases refine the expert states by geometric context. The protocol covers every valid execution and may also admit additional observation sequences. Write $d \in \{0, 1\}$. An edge $x : j$ reads x and advances to phase j; $d : j$ permits either digit. Expert state g_i supplies the write and motion from Section 4.2. A dash denotes a terminal phase with no outgoing transition.

i	Phase meaning	g_i	Allowed reading : next phase
0	First input digit	0	$d : 1$
1	Seek the separator	0	$d : 1$ $+ : 2$
2	Initial descent	1	$d : 3$
3	Place the marker	1	$\lambda : 4$
4	Ensure an accumulator digit	8	$d : 6$ $\lambda : 5$
5	Digit after zero extension	2	$d : 6$
6	Scan toward separator	2	$d : 6$ $+ : 7$
7	First cell after separator	2	$d : 8$ $\lambda : 22$
8	Scan remaining right operand	2	$d : 8$ $\lambda : 9$
9	Read rightmost factor digit	3	$0 : 10$ $1 : 14$
10	Transport zero: first blank	4	$\lambda : 11$
11	Transport zero: second blank	4	$\lambda : 12$
12	Transport zero: walk left	4	$\lambda : 12$ $+ : 13$
13	Add zero above marker	4	$d : 21$
14	Transport one: first blank	5	$\lambda : 15$
15	Transport one: second blank	5	$\lambda : 16$
16	Transport one: walk left	5	$\lambda : 16$ $+ : 17$
17	Add one above marker	5	$0 : 21$ $1 : 18$
18	Propagate a carry	6	$1 : 18$ $0 : 19$ $\lambda : 19$
19	First return cell after carry	7	$\lambda : 20$
20	Return to the marker	7	$\lambda : 20$ $+ : 3$
21	At marker, without carry	7	$+ : 3$
22	Erase the input separator	3	$+ : 23$
23	First cleanup cell	9	$\lambda : 24$
24	Find and erase final marker	9	$\lambda : 24$ $+ : 25$
25	Preserve result and stop	9	$d : 26$
26	Halted	10	–

Lemma 3 (Protocol Coverage)**.** *Every execution of $\mathcal{E}$ from a valid input follows this protocol, starting at phase* 0 *and ending at phase* 26*.*

Proof. The nonempty first operand establishes phases 0–3. Lemma 1 places a digit immediately after a zero extension in phase 4, establishing phase 5. The right scan meets the separator before any blank. To the separator's right, a first blank means the next left step reads the separator (phase 22); after any digit, that step reads a digit (phase 9). Equation (1) guarantees two initial blank readings while transporting either bit. The cell above the marker contains a digit. A carry ends strictly left of the marker, so the first return cell contains a blank. A noncarry addition returns directly to the marker. Erasing and shifting the marker reaches a blank. Finally, cleanup starts below the separator on a blank, reaches the marker and then a digit. These facts justify every branch of the table. Induction over the steps and rounds proves coverage; Lemma 2 supplies the final phase. □

4 Simulation Certificate

4.1 Phase Relation and Local Checks

For phase i, write Q_i for the controller-state set in the following table. The relation pairs an expert configuration in phase i with a controller configuration having the *same grid and head* and a state in Q_i.

Phase i	Meaning	Expert state g_i	Controller states Q_i
0	First input digit	0	{0}
1	Seek the separator	0	{1}
2	Initial descent	1	{2}
3	Place the marker	1	{3}
4	Ensure an accumulator digit	8	{4}
5	Digit after zero extension	2	{15}
6	Scan toward separator	2	{5}
7	First cell after separator	2	{6}
8	Scan remaining right operand	2	{6}
9	Read rightmost factor digit	3	{7}
10	Transport zero: first blank	4	{12}
11	Transport zero: second blank	4	{13}
12	Transport zero: walk left	4	{13}
13	Add zero above marker	4	{14}
14	Transport one: first blank	5	{8}
15	Transport one: second blank	5	{9}
16	Transport one: walk left	5	{9}
17	Add one above marker	5	{10}
18	Propagate a carry	6	{18}
19	First return cell after carry	7	{19, 20}
20	Return to the marker	7	{20}
21	At marker, without carry	7	{11}
22	Erase the input separator	3	{7}
23	First cleanup cell	9	{16}
24	Find and erase final marker	9	{16}
25	Preserve result and stop	9	{17}
26	Halted	10	{21}

Local Transition Identity. For every listed $q \in Q_i$ and every allowed edge $i \xrightarrow{x} j$ from the protocol, the actual tables satisfy

$$\delta_{\mathcal{E}}(g_i, x) = (g_j, w, M), \qquad \delta_C(q, x) = (q', w, M), \qquad q' \in Q_j. \tag{2}$$

Every required controller entry exists. A transition reaches the halt state exactly when $j = 26$, with motion S. The certificate contains **28 phase–state pairs** (including the terminal pair) and **50 local checks**. The checks cover 43 distinct nonterminal controller entries. Section 4.2 provides the tables for direct inspection; the accompanying script checks every instance of (2).

4.2 Transition Tables

Each entry lists (next state, written symbol, motion). The symbol λ means blank; a dash means an undefined transition. The tables reproduce the adjacent exports with the original state numbers.

Expert: `add_grid(2)`. Initial state 0; halt 10.

State	λ	+	0	1
0	$(10, \lambda, \mathsf{S})$	$(1, +, \mathsf{L})$	$(0, 0, \mathsf{R})$	$(0, 1, \mathsf{R})$
1	$(8, +, \mathsf{U})$	$(10, +, \mathsf{S})$	$(1, 0, \mathsf{D})$	$(1, 1, \mathsf{D})$
2	$(3, \lambda, \mathsf{L})$	$(2, +, \mathsf{R})$	$(2, 0, \mathsf{R})$	$(2, 1, \mathsf{R})$
3	$(10, \lambda, \mathsf{S})$	$(9, \lambda, \mathsf{D})$	$(4, \lambda, \mathsf{D})$	$(5, \lambda, \mathsf{D})$
4	$(4, \lambda, \mathsf{L})$	$(4, +, \mathsf{U})$	$(7, 0, \mathsf{D})$	$(7, 1, \mathsf{D})$
5	$(5, \lambda, \mathsf{L})$	$(5, +, \mathsf{U})$	$(7, 1, \mathsf{D})$	$(6, 0, \mathsf{L})$
6	$(7, 1, \mathsf{D})$	$(10, +, \mathsf{S})$	$(7, 1, \mathsf{D})$	$(6, 0, \mathsf{L})$
7	$(7, \lambda, \mathsf{R})$	$(1, \lambda, \mathsf{L})$	$(10, 0, \mathsf{S})$	$(10, 1, \mathsf{S})$
8	$(2, 0, \mathsf{R})$	$(10, +, \mathsf{S})$	$(2, 0, \mathsf{R})$	$(2, 1, \mathsf{R})$
9	$(9, \lambda, \mathsf{L})$	$(9, \lambda, \mathsf{U})$	$(10, 0, \mathsf{S})$	$(10, 1, \mathsf{S})$
10	$(10, \lambda, \mathsf{S})$	$(10, +, \mathsf{S})$	$(10, 0, \mathsf{S})$	$(10, 1, \mathsf{S})$

Metric-Extracted Controller. Initial state 0; halt 21.

State	λ	+	0	1
0	–	–	$(1, 0, \mathsf{R})$	$(1, 1, \mathsf{R})$
1	–	$(2, +, \mathsf{L})$	$(1, 0, \mathsf{R})$	$(1, 1, \mathsf{R})$
2	–	–	$(3, 0, \mathsf{D})$	$(3, 1, \mathsf{D})$
3	$(4, +, \mathsf{U})$	–	–	–
4	$(15, 0, \mathsf{R})$	–	$(5, 0, \mathsf{R})$	$(5, 1, \mathsf{R})$
5	–	$(6, +, \mathsf{R})$	$(5, 0, \mathsf{R})$	$(5, 1, \mathsf{R})$
6	$(7, \lambda, \mathsf{L})$	–	$(6, 0, \mathsf{R})$	$(6, 1, \mathsf{R})$
7	–	$(16, \lambda, \mathsf{D})$	$(12, \lambda, \mathsf{D})$	$(8, \lambda, \mathsf{D})$
8	$(9, \lambda, \mathsf{L})$	–	–	–
9	$(9, \lambda, \mathsf{L})$	$(10, +, \mathsf{U})$	–	–
10	–	–	$(11, 1, \mathsf{D})$	$(18, 0, \mathsf{L})$
11	–	$(3, \lambda, \mathsf{L})$	–	–
12	$(13, \lambda, \mathsf{L})$	–	–	–
13	$(13, \lambda, \mathsf{L})$	$(14, +, \mathsf{U})$	–	–
14	–	–	$(11, 0, \mathsf{D})$	$(11, 1, \mathsf{D})$
15	–	–	$(5, 0, \mathsf{R})$	$(5, 1, \mathsf{R})$
16	$(16, \lambda, \mathsf{L})$	$(17, \lambda, \mathsf{U})$	–	–
17	–	–	$(21, 0, \mathsf{S})$	$(21, 1, \mathsf{S})$
18	$(20, 1, \mathsf{D})$	–	$(19, 1, \mathsf{D})$	$(18, 0, \mathsf{L})$
19	$(20, \lambda, \mathsf{R})$	–	–	–
20	$(20, \lambda, \mathsf{R})$	$(3, \lambda, \mathsf{L})$	–	–
21	$(21, \lambda, \mathsf{S})$	$(21, +, \mathsf{S})$	$(21, 0, \mathsf{S})$	$(21, 1, \mathsf{S})$

5 Equivalence Proof

Proof of Theorem 1. Initially both configurations have the same grid and head, with control states 0 related by phase 0. Suppose the relation holds before a controller step. Lemma 3 supplies an allowed read symbol x. Equal grids and heads give the same read symbol to both controllers. Equation (2) gives equal writes and motions and preserves the relation between successor states. The next grids and heads therefore agree.

Induction establishes equality of every finite execution prefix. The terminal condition permits a halt exactly on an edge to the terminal phase. Lemma 2 establishes termination. Both controllers therefore halt together with equal complete PTOs and final configurations. □

Defined Transitions. The metric controller splits expert states according to position in the algorithm. For example, expert state 4 (transport and add zero) corresponds to controller states 12, 13, 14, while expert state 9 (cleanup) corresponds to 16, 17. The phase invariant excludes every missing entry on valid inputs. The certificate checks all 43 defined nonterminal entries. The remaining four entries contain halt loops. The theorem requires no completion of undefined transitions.

6 Verification and Artifacts

Verification. Run `python3 verify_equivalence.py --quick` from this proof directory to verify the source fingerprints, equality of the CSV and JSON controller exports, equality of the exported expert with `add_grid(2)`, and every local certificate obligation. Omitting `--quick` also compares two independent grid executions on 15,876 exhaustive operand pairs (all lengths 1–6), 44 directed cases up to 1024 digits, and 128 fixed-seed random cases. All 4,766,713 compared transitions, final grids and head positions agree. These finite tests supplement the universal proof; Lemmas 1–3 and equation (2) establish the length-independent claim.

Artifact Identity. The title page gives the repository-relative case directory. This proof directory contains the controller and expert exports, reference code, protocol, checker, and saved verification report. The verification script stores or reads SHA-256 checksums for these artifacts. A checksum identifies the exact bytes of a file; the checker compares the stored and computed values to detect changes. These checks bind the certificate to the supplied exports. The invariants, protocol coverage, and local transition identity establish equivalence.